MLP Coursework 1

S2890639

Abstract

In this report we study the problem of overfitting, which is the training regime where performance increases on the training set but decrease on validation data. Overfitting prevents our trained model from generalizing well to unseen data. We first analyse the given example and discuss the probable causes of the underlying problem. Then we investigate how the depth and width of a neural network can affect overfitting in a feedforward architecture and observe that increasing width and depth tend to enable further overfitting. Next we discuss how two standard methods, Dropout and Weight Penalty, can mitigate overfitting, then describe their implementation and use them in our experiments to reduce the overfitting on the EMNIST dataset. Based on our results, we ultimately find that both dropout and weight penalty are able to mitigate overfitting.

We further explore alternative loss and activation functions, and evaluate whether their performance relates to problems of overfitting.

Finally, we conclude the report with our observations and related work. Our main findings indicate that preventing overfitting is achievable through regularization, although combining different methods together is not straightforward.

1. Introduction

In this report we focus on a common and important problem while training machine learning models known as overfitting, or overtraining [1], which is the training regime where performances increase on the training set but decrease on unseen data. We first start with analysing the given problem in Figure 1, study it in different architectures and then investigate different strategies to mitigate the problem. In particular, Section 2 identifies and discusses the given problem, and investigates the effect of network width and depth in terms of generalization gap (see Ch. 5 in Goodfellow et al. 2016) and generalization performance. Section 3 introduces two regularization techniques to alleviate overfitting: Dropout (Srivastava et al., 2014) and L1/L2 Weight Penalties (see Section 7.1 in Goodfellow et al. 2016). We first explain them in detail and discuss why they are used for alleviating overfitting. We extend our study to implement a combined L1 and L2 penalty.

In Section 4, we incorporate each of them and their various combinations to a three hidden layer[2] neural network, train it on the EMNIST dataset, which contains 131,600 images of characters and digits, each of size 28x28, from 47 classes. We evaluate them in terms of generalization gap and performance, and discuss the results and effectiveness of the tested regularization strategies. Our results show that both dropout and weight penalty are able to mitigate overfitting. We end Section 4, by testing an alternative activation function, and explore whether the observed performance degradation relates to overfitting or some other problem.

Finally, we conclude our study in Section 5, noting that preventing overfitting is achievable through regularization, although combining different methods together is not straightforward.

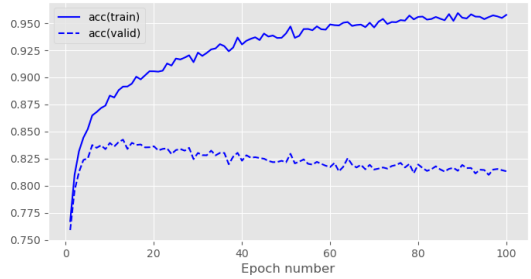
2. Problem identification

Overfitting to training data is a very common and important issue that needs to be dealt with when training neural networks or other machine learning models in general (see Ch. 5 in Goodfellow et al. 2016). A model is said to be overfitting when as the training progresses, its performance on the training data keeps improving, while its is degrading on validation data. Effectively, the model stops learning related patterns for the task and instead starts to memorize specificities of each training sample that are irrelevant to new samples. Overfitting leads to bad generalization performance in unseen data, as performance on validation data is indicative of performance on test data and (to an extent) during deployment.

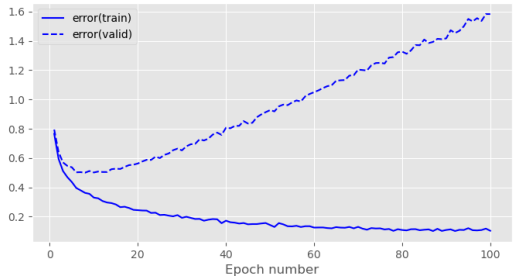
Although it eventually happens to all gradient-based training, it is most often caused by models that are too large with respect to the amount and diversity of training data. The more free parameters the model has, the easier it will be to memorize complex data patterns that only apply to a restricted amount of samples. A prominent symptom of overfitting is the generalization gap, defined as the difference between the validation and training error. A steady increase in this quantity is usually interpreted as the model

[1] Technically, overtraining is the act of training beyond a point at which performance degrades; while overfitting is training to the extent that the learnt function is more complex than required to capture the training data. They correlate, but they are not, strictly speaking, the same thing. To keep things simple, and according to common usage of the terms, we are using them interchangeably in this document, and referring to their joint effect.

[2] We denote all layers as hidden except the final (output) one. This means that depth of a network is equal to the number of its hidden layers + 1.



(a) accuracy by epoch



(b) error by epoch

Figure 1. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for the baseline model.

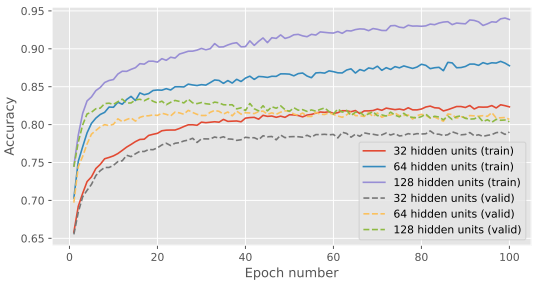
entering the overfitting regime.

Figure 1a and 1b show a prototypical example of overfitting. We see in Figure 1a that early in training (epochs 0-5), both training and validation accuracies in Figure 1a have high positive velocities, rising steeply. This behavior is mirrored inversely in Figure 1b by training and validation errors, which both have large negative velocities at this stage, indicating rapid fit. From around epoch 10, the validation accuracy curves show smaller magnitudes and their velocities decay toward zero as learning saturates. This flattening is visible in Figure 1a, where the validation curve's slope decreases steadily between epochs 10-18 before reaching its peak. In contrast, the training accuracy slope decreases but remains consistently positive throughout this period, indicating continued improvement. Around epoch 20, the velocity of validation accuracy crosses from positive to negative (the curve peaks), while the validation error velocity crosses from negative to positive (the curve bottoms out). This mirrors the accuracy behaviour exactly: in Figure 1b, validation error reaches its minimum at the same point the validation accuracy reaches its maximum. Meanwhile, training velocities in both Figures remain favorable. This divergence of training and validation performance, when the validation slopes change in sign and the training slopes remains favourable, is what indicates overfitting.

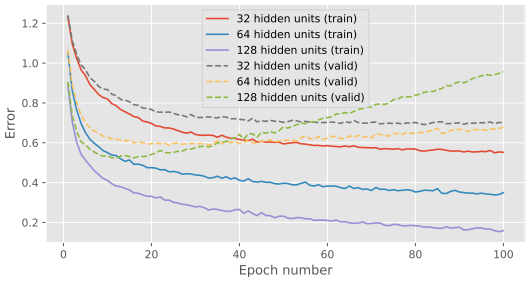
The extent to which our model overfits depends on many factors. For example, the quality and quantity of the training set and the complexity of the model. If we have sufficiently many diverse training samples, or if our model contains few hidden units, it will in general be less prone to overfitting.

# Hidden Units	Val. Acc.	Train Error	Val. Error
32	79.0	0.552	0.700
64	80.7	0.350	0.807
128	80.4	0.159	0.958

Table 1. Validation accuracy (%) and training/validation error (in terms of cross-entropy error) for varying network widths on the EMNIST dataset.



(a) accuracy by epoch



(b) error by epoch

Figure 2. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for different network widths.

Any form of regularization will also limit the extent to which the model overfits.

2.1. Network width

First we investigate the effect of increasing the number of hidden units in a single hidden layer network when training on the EMNIST dataset. The network is trained using the Adam optimizer with a learning rate of 10^{-3} and a batch size of 100, for a total of 100 epochs.

The input layer is of size 784, and output layer consists of 47 units. Three different models were trained, with a single hidden layer of 32, 64 and 128 ReLU hidden units respectively. Figure 2 depicts the error and accuracy curves over 100 epochs for the model with varying number of hidden units. Table 1 reports the final accuracy, training error, and validation error. We observe that it shows that the validation accuracy improves only marginally with increased network width—rising by 2.2% from 32 → 64 units and decreasing slightly by -0.3% from 64 → 128 units. As seen in Figure 2a, the accuracy curves for wider networks exhibit a higher positive velocity during the first 10–15 epochs,

rising more steeply than the 32-unit model and indicating faster initial learning. After this early phase, the validation accuracy velocities for the 64- and 128-unit models gradually decline towards zero as the curves flatten, and eventually change sign around epoch 18 when the validation curves peak. In contrast, the 32-unit model's validation curve maintains a small positive velocity for longer and seems to be plateauing at around epoch 100.

A similar velocity pattern is seen in Figure 2b. The training error curves for the wider models display a large negative velocity through epochs 10–15, reflecting rapid error reduction. Their validation error curves also initially have negative velocity but reach their minimum earlier, after which their velocities change from negative to positive, indicating that the model begins to overfit. The 32-unit model shows a more gradual decay in validation error velocity that approaches zero without sharply reversing.

Numerically, training error decreases by -36.5% and -54.6% when increasing the width from 32→64 and 64→128 units, respectively, but validation error increases by 15.3% when going from 32→64 units, and 18.7% when going from 64→128 units. This widening generalisation gap, together with the opposing validation and training velocities, demonstrates diminishing returns at higher widths: the additional capacity drives continued improvement on the training set while harming validation performance.

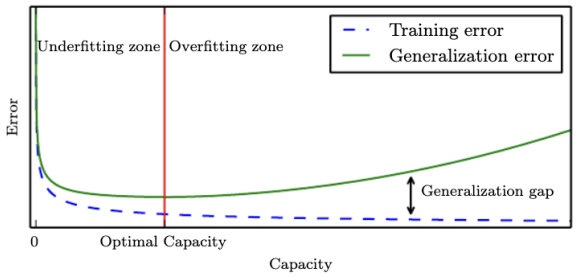


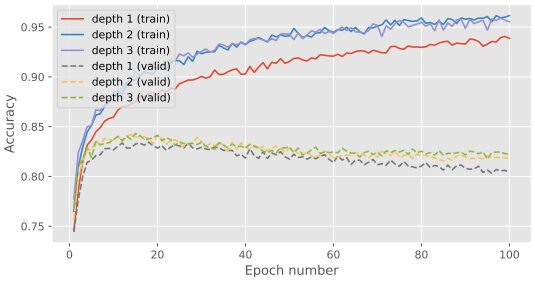
Figure 3. Bias–variance trade-off (Reproduced from Figure 5.3 in (Goodfellow et al., 2016)). As model capacity increases, training error decreases monotonically while validation error eventually rises due to overfitting.

This behaviour matches the classical bias–variance trade-off described in Chapter 5 of Deep Learning by Goodfellow et al. (Goodfellow et al., 2016), and illustrated in Figure 3. As width increases, model capacity rises, leading to monotonically decreasing training error but worsening validation error beyond a certain point, which is exactly what we observe in Figure 2b for the 64- and 128-unit models. The pattern is consistent across widths: as the network becomes larger, the generalisation gap widens.

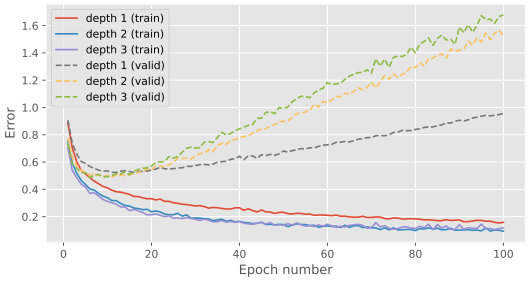
The minimal gains in validation accuracy and the rise in validation error at higher widths indicate overfitting. This implies that overly large models can memorise training data without improving generalisation. In contrast, the 32-unit model demonstrates a more favourable balance: its validation accuracy continues to improve until it plateaus, but it does not worsen as observed for wider models. Overall, the results align with theoretical predictions while clearly illustrating the trade-off between model capacity and generalisation .

# Hidden Layers	Val. Acc.	Train Error	Val. Error
1	80.4	0.159	0.958
2	81.8	0.093	1.530
3	82.3	0.117	1.680

Table 2. Validation accuracy (%) and training/validation error (in terms of cross-entropy error) for varying network depths on the EMNIST dataset.



(a) accuracy by epoch



(b) error by epoch

Figure 4. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for different network depths.

2.2. Network depth

Next we investigate the effect of varying the number of hidden layers in the network. Table 2 and Figure 4 depict results from training three models with one, two and three hidden layers respectively, each with 128 ReLU hidden units. As with previous experiments, they are trained with the Adam optimizer with a learning rate of 9×10^{-4} and a batch size of 100.

We observe that validation accuracy increases modestly with depth, rising by 1.7% from 1→2 layers and by a further 0.6% from 2→3 layers (Table 2). Figure 4a shows that deeper networks exhibit steeper initial accuracy velocities during epochs 0–10: both the 2- and 3-layer models rise more sharply than the 1-layer model, indicating faster early convergence. Between epochs 10–15, the validation curves

begin to flatten as their velocities approach zero, and around epoch 20 their slopes change sign, marking the peak validation accuracy. Beyond this point, the validation accuracy decreases with negative velocity.

This behaviour is mirrored in the error curves in Figure 4b. Validation error decreases rapidly during epochs 0–10 (large negative velocity), approaches a minimum between epochs 10–15, and then increases after epoch 20 as its velocity becomes positive. Training curves follow a similar but consistently more favourable trajectory: deeper networks show stronger negative training-error velocities up to roughly epoch 20 and converge to lower final training errors ($0.159 \rightarrow 0.093 \rightarrow 0.117$) before plateauing. This indicates that additional depth improves the model's ability to fit the training data, although the benefit saturates beyond two layers.

However, these improvements do not translate into better generalisation. Validation error increases by 59.7% from 1→2 layers and by 9.8% from 2→3 layers, and the validation curves diverge from the training curves shortly after epoch 20. This indicates earlier and stronger overfitting in deeper models, validating that the one layer model suffers the least from overfitting.

Increasing depth affects the results in a consistent and largely expected manner. Training error decreases substantially when moving from 1 to 2 layers ($0.159 \rightarrow 0.093$), but the improvement saturates at 3 layers, where the training error increases slightly to 0.117. This behaviour aligns with theoretical expectations that additional depth increases representational capacity but can introduce optimisation difficulties or diminishing returns in small fully connected networks.

At the same time, the worsening validation error and the earlier divergence between training and validation curves indicate that deeper networks exhibit higher variance. This matches the classical capacity–generalisation relationship described by Goodfellow et al., (Goodfellow et al., 2016): as depth increases, models become more expressive and more capable of fitting the training data, but also more prone to memorising training-specific patterns without sufficient regularisation or data.

The limited improvement in validation accuracy, together with the substantial rise in validation error for the 2-layer and 3-layer models, therefore reflects exactly the trend predicted by theory: increased depth improves representation capacity but degrades generalisation in this setting. Overall, the empirical results are consistent with expectations from the literature.

3. Regularization

In this section, we investigate three regularization methods to alleviate the overfitting problem, specifically dropout layers, the L1 and L2 weight penalties and label smoothing.

3.1. Dropout

Dropout (Srivastava et al., 2014) is a stochastic method that randomly inactivates neurons in a neural network according to an hyperparameter, the inclusion rate (*i.e.* the rate that an unit is included). Dropout is commonly represented by an additional layer inserted between the linear layer and activation function. Its forward pass during training is defined as follows:

$$\text{mask} \sim \text{bernoulli}(p) \tag{1}$$

$$\boldsymbol{y}' = \text{mask} \odot \boldsymbol{y} \tag{2}$$

where $\boldsymbol{y}, \boldsymbol{y}' \in \mathbb{R}^d$ are the output of the linear layer before and after applying dropout, respectively. mask $\in \mathbb{R}^d$ is a mask vector randomly sampled from the Bernoulli distribution with inclusion probability p, and $\odot$ denotes the element-wise multiplication.

At inference time, stochasticity is not desired, so no neurons are dropped. To account for the change in expectations of the output values, we scale them down by the inclusion probability p:

$$\boldsymbol{y}' = \boldsymbol{y} * p \tag{3}$$

As there is no nonlinear calculation involved, the backward propagation is just the element-wise product of the gradients with respect to the layer outputs and mask created in the forward calculation. The backward propagation for dropout is therefore formulated as follows:

$$\frac{\partial \boldsymbol{y}'}{\partial \boldsymbol{y}} = mask \tag{4}$$

Dropout is an easy to implement and highly scalable method. It can be implemented as a layer-based calculation unit, and be placed on any layer of the neural network at will. Dropout can reduce the dependence of hidden units between layers so that the neurons of the next layer will not rely on only few features from of the previous layer. Instead, it forces the network to extract diverse features and evenly distribute information among all features. By randomly dropping some neurons in training, dropout makes use of a subset of the whole architecture, so it can also be viewed as bagging different sub networks and averaging their outputs.

3.2. Weight penalty

L1 and L2 regularization (Ng, 2004) are simple but effective methods to mitigate overfitting to training data. The application of L1 and L2 regularization strategies could be formulated as adding penalty terms with L1 and L2 norm square of weights in the cost function without changing other formulations. The idea behind this regularization method is to penalize the weights by adding a term to the cost function, and explicitly constrain the magnitude of the weights with either the L1 and L2 norms. The optimization

problem takes a different form:

$$\text{L1: } \min_{\boldsymbol{w}} E_{\text{data}}(\boldsymbol{X}, \boldsymbol{y}, \boldsymbol{w}) + \lambda \|w\|_1 \tag{5}$$

$$\text{L2: } \min_{\boldsymbol{w}} E_{\text{data}}(\boldsymbol{X}, \boldsymbol{y}, \boldsymbol{w}) + \lambda \|w\|_2^2 \tag{6}$$

where E_{data} denotes the cross entropy error function, and $\{\boldsymbol{X}, \boldsymbol{y}\}$ denotes the input and target training pairs. λ controls the strength of regularization.

Weight penalty works by constraining the scale of parameters and preventing them to grow too much, avoiding overly sensitive behaviour on unseen data. While L1 and L2 regularization are similar to each other in calculation, they have different effects. Gradient magnitude in L1 regularization does not depend on the weight value and tends to bring small weights to 0, which can be used as a form of feature selection, whereas L2 regularization tends to shrink the weights to a smaller scale uniformly.

3.2.1. Combination of L1 and L2 Regularisation

According to Zou & Hastie 2005, combining L1 and L2 regularisation can leverage the strengths of both methods to improve model performance. This technique is often referred to as Elastic Net regularisation. The combined loss function can be expressed as:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_1 \sum_i |w_i| + \lambda_2 \sum_i w_i^2 \tag{7}$$

where:

- $\mathcal{L}$ is the total loss function.
- $\mathcal{L}_{\text{data}}$ is the original loss function (e.g., cross-entropy loss).
- w_i represents the weights of the model.
- λ_1 is the hyperparameter controlling the strength of L1 regularisation, promoting sparsity in the model weights.
- λ_2 is the hyperparameter controlling the strength of L2 regularisation, encouraging smaller weight values and reducing overfitting.

The potential benefits of this approach include:

- Sparsity: L1 regularisation can lead to sparse models by driving some weights to zero, effectively performing feature selection.
- Stability: L2 regularisation helps to stabilize the model by penalizing large weights, which can improve generalization.
- Flexibility: The combination allows for a balance between sparsity and stability, which can be tuned based on the specific dataset and task.

In our implementation, we can use a mixing parameter α to control the relative contributions of L1 and L2 regularisation:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \left(\alpha \sum_i |w_i| + (1 - \alpha) \sum_i w_i^2 \right) \tag{8}$$

where λ is the overall regularisation strength, and α (ranging from 0 to 1) determines the balance between L1 and L2 penalties. By controlling α, we can adjust the model to prioritize either sparsity or stability, depending on the requirements of the task at hand.

3.3. Label smoothing

Label smoothing regularizes a model based on a softmax with K output values by replacing the hard target 0 labels and 1 labels with $\frac{\alpha}{K-1}$ and $1 - \alpha$ respectively. α is typically set to a small number such as 0.1.

$$\begin{cases} \frac{\alpha}{K-1}, & \text{if} \quad t_k = 0 \\ 1 - \alpha, & \text{if} \quad t_k = 1 \end{cases} \tag{9}$$

The standard cross-entropy error is typically used with these *soft* targets to train the neural network. Hence, implementing label smoothing requires only modifying the targets of training set. This strategy may prevent a neural network to obtain very large weights by discouraging very high output values.

4. Balanced EMNIST Experiments

Here we evaluate the effectiveness of the given regularization methods for reducing the overfitting on the EMNIST dataset. We build a baseline architecture with three hidden layers, each with 128 neurons, which suffers from overfitting as shown in section 2.

Here we train the network with a lower learning rate of 10^{-4}, as the previous runs were overfitting after only a handful of epochs. Results for the new baseline (c.f. Table 3) confirm that lower learning rate helps, so all further experiments are run using it.

Here, we apply the L1 or L2 regularization with dropout to our baseline and search for good hyperparameters on the validation set. Then, we apply the label smoothing with $\alpha = 0.1$ to our baseline. We summarize all the experimental results in Table 3. For each method except the label smoothing, we plot the relationship between generalization gap and validation accuracy in Figure 5.

First we analyze three methods separately, train each over a set of hyperparameters and compare their best performing results.

The results from L1 and L2 regularization methods provided in Table 3 show that the two penalties behave very differently with respect to λ. For L1, the best-performing value is $\lambda = 5 \times 10^{-4}$, which yields the highest validation accuracy (79.5%) and the lowest validation error (0.658) among the stable runs. Increasing λ further leads to rapid

Model	Hyperparameter value(s)	Validation accuracy	Train Error	Validation Error
Baseline	-	83.7	0.241	0.533
Dropout	0.6	80.7	0.549	0.593
	0.7	83.1	0.448	0.509
	0.85	85.1	0.329	0.434
	0.97	***85.4***	***0.244***	***0.457***
L1 penalty	5e-4	*79.5*	*0.642*	*0.658*
	1e-3	74.3	0.879	0.880
	5e-3	2.41	3.850	3.850
	5e-2	2.20	3.850	3.850
L2 penalty	5e-4	*85.1*	*0.306*	*0.460*
	1e-3	84.8	0.358	0.453
	5e-3	81.3	0.586	0.607
	5e-2	39.2	2.258	2.256
Label smoothing	0.1	84.9	1.02	1.16

Table 3. Results of all hyperparameter search experiments. *italics* indicate the best results per series (Dropout, L1 Regularisation, L2 Regularisation, Label smoothing) and **bold** indicates the best overall.

Run	λ / α	Val. Acc. (%)	Train Error	Val. Error
1	5×10^{-5} / 0.900	85.1	0.309	0.455
2	5×10^{-5} / 0.750	85.1	0.305	0.455
3	5×10^{-5} / 0.500	85.0	0.284	0.459
4	5×10^{-5} / 0.250	84.2	0.277	0.488

Table 4. Validation accuracy (%) and training/validation cross-entropy error for four runs with a fixed regularisation strength $\lambda = 5 \times 10^{-5}$ and varying Elastic Net mixing parameter α.

degradation: at $\lambda \geq 10^{-3}$ the model already performs significantly worse, and at $\lambda \geq 5 \times 10^{-3}$ optimisation collapses entirely. This indicates that L1 is highly sensitive and tends to function well at very small regularisation strengths.

For L2, the optimal region is broader. Validation accuracy is highest at $\lambda = 5 \times 10^{-4}$ (85.1%), while validation error is lowest at $\lambda = 10^{-3}$ (0.453). Suggesting that L2 performs best with moderate regularisation ranging from 5×10^{-4}–10^{-3}, and is far more tolerant than L1. However, at $\lambda = 5 \times 10^{-2}$ even L2 collapses, showing that excessively large penalties remain undesirable.

Although L2 performs best at moderate λ, the strong sensitivity of L1 means that any λ chosen in this middle region would cause the L1 term to dominate the combined Elastic Net penalty. In order to study the interaction between the L1 and L2 components, we require a λ small enough that both penalties contribute, and does not overwhelm optimisation, so that the effect of varying α can be observed cleanly.

This motivates the following research question:

> *When λ is small enough that neither regulariser dominates, does shifting the Elastic Net mixture toward L1 or toward L2 lead to better generalisation on EMNIST?*

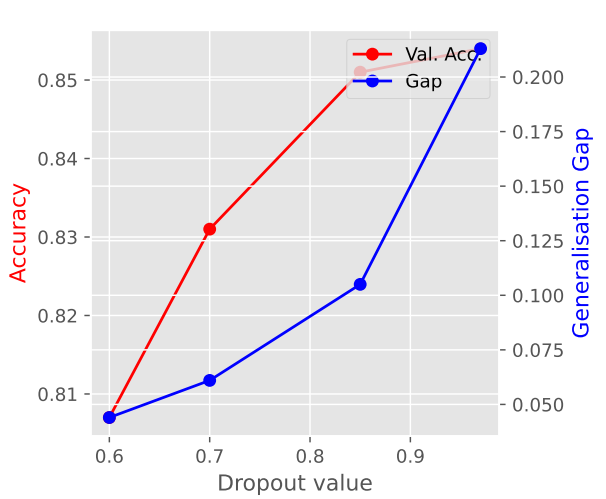
To answer this, we fix a conservative value of $\lambda = 5 \times 10^{-5}$, which ensures stable training while preserving meaningful contributions from both penalties across α. Holding λ fixed also isolates α as the sole factor controlling the balance: L1 promotes sparsity while L2 provides smooth weight shrinkage. Varying α therefore directly probes the interaction between these two effects.

Given the budget of four experiments, we select α values spanning the L2-dominant to L1-dominant regimes:

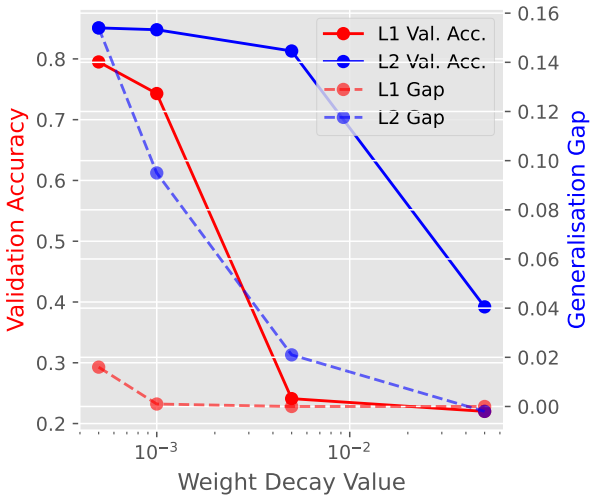
- Run 1: $\lambda = 5 \times 10^{-5}$, $\alpha = 0.90$ (strong L1-dominant)
- Run 2: $\lambda = 5 \times 10^{-5}$, $\alpha = 0.75$ (moderately L1-dominant)
- Run 3: $\lambda = 5 \times 10^{-5}$, $\alpha = 0.50$ (balanced mixture)
- Run 4: $\lambda = 5 \times 10^{-5}$, $\alpha = 0.25$ (L2-dominant)

These configurations form a structured sweep across the full mixture range and allow us to assess how the relative weighting of sparsity (L1) and shrinkage (L2) influences generalisation performance when the overall regularisation strength is held constant.

On another note, to identify the best hyperparameter configuration, we performed a four-run sweep for additional regularisation techniques such as Dropout, L1, L2, in addition to the Elastic Net experiments described earlier. Apart

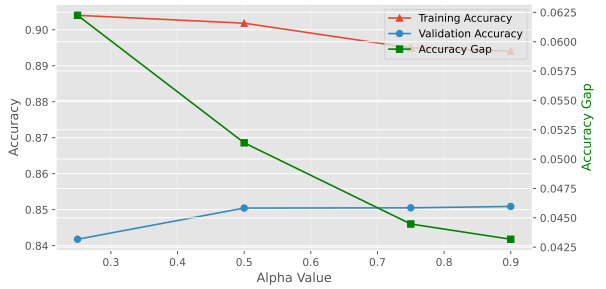


(a) Accuracy and error by inclusion probability.

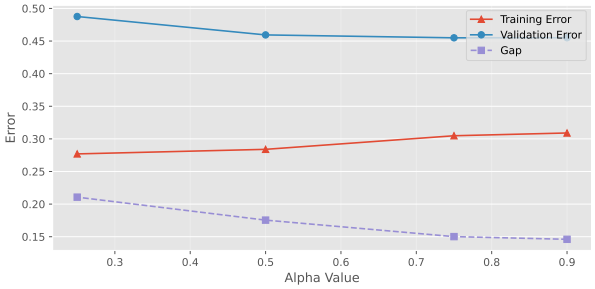


(b) Accuracy and error by weight penalty.

Figure 5. Accuracy and error by regularisation strength of each method (Dropout and L1/L2 Regularisation).



(a) Validation and training accuracy, and generalisation gap for l1+l2 weight penalty.



(b) Training and validation error, and generalisation gap for l1+l2 weight penalty.

Figure 6. Training and validation performance across regularisation strengths for L1/L2 methods. The generalisation gap highlights overfitting effects across experiments.

from that, we did a single run with label smoothing for comparison. All runs were trained for 100 epochs with a learning rate of 1×10^{-4}, width 128, depth 4, and batch size 100, using Adam optimisation with Cross-Entropy loss on the EMNIST dataset. These settings were selected based on prior experiments, which indicated stable convergence without overfitting.

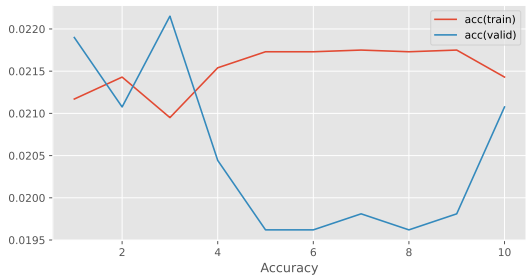
For Dropout, we evaluated inclusion probabilities of 0.6, 0.7, 0.85, and 0.97, spanning a spectrum from strong to weak regularisation. For label smoothing, we tested a smoothing factor of 0.1, a standard choice that balances confidence and uncertainty in the target distribution. Overall, all regularisation techniques except L1 by itself improved validation accuracy relative to the baseline model.

Dropout According to Table 3, the best-performing configuration (and best result overall) was Dropout with an inclusion probability of 0.97. It achieved a validation accuracy of 85.4%, a training error of 0.244, and a validation error of 0.457. This configuration also exhibits the second-lowest validation error among all tested Dropout levels.

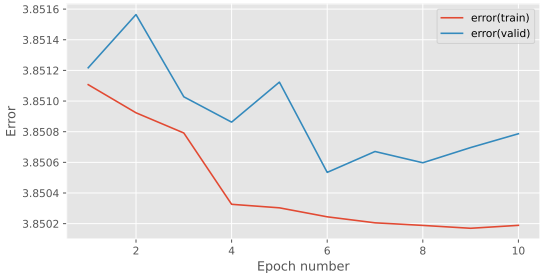
Figure 5a illustrates a consistent trend: increasing the inclusion probability improves validation accuracy while simultaneously widening the generalisation gap. This indicates that although the model fits the training data more closely at high inclusion rates, it also retains enough regularisation to prevent overfitting, as the validation error does not increase substantially.

Label Smoothing Label smoothing achieved a validation accuracy of 84.9%, with a training error of 1.02 and a validation error of 1.16. While this represents an improvement over the baseline, it is clearly outperformed by both Dropout and L2 regularisation. The relatively high validation error suggests that smoothing introduced excessive uncertainty for this task. However, the generalisation gap decreased substantially from 0.292 $\rightarrow$ 0.140, indicating that label smoothing effectively mitigates model overconfidence even when accuracy gains are limited.

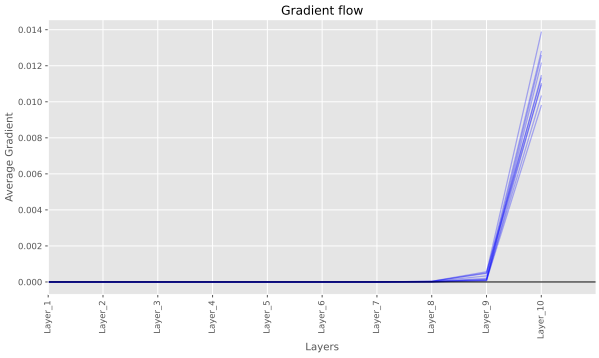
L1 and L2 Regularisation L1 regularisation performed poorly across all tested strengths. The most competitive configuration ($\lambda = 5 \times 10^{-4}$) achieved a validation accuracy



(a) Validation and training accuracy, for custom activation layer.



(b) Training and validation error, and generalisation gap for custom activation layer.



(c) Training and validation gradients for custom activation layer.

Figure 7. Training and validation performance across regularisation strengths for custom activation layer.

of only 79.5%, and larger λ values caused optimisation collapse. This behaviour is consistent with the nature of L1: inducing sparsity may remove too many informative features, leading to underfitting. Figure 5b supports this interpretation, showing that increasing L1 strength consistently worsens validation accuracy and enlarges the generalisation gap before optimisation fails entirely.

In contrast, L2 regularisation delivered strong results. The best L2 configuration ($\lambda = 5 \times 10^{-4}$) achieved a validation accuracy of 85.1%, a training error of 0.306, and a validation error of 0.460. This small λ appears to provide the right balance: sufficient shrinkage to reduce overfitting, without impeding the model's ability to learn. As seen in Figure 5b, increasing L2 strength initially improves performance, but excessive regularisation eventually leads to optimisation difficulties.

Elastic Net Regularisation We now turn to the Elastic Net experiments summarised in Table 4 and Figure 6. The trends mirror those observed for L1 and L2 individually: higher α values (i.e. a stronger L1 component) lead to lower accuracy and error gaps and higher validation accuracy. This indicates that sparsity contributes positively to generalisation at the tested λ level.

Run 2 ($\alpha = 0.75$) emerged as the best-performing configuration, with a validation accuracy of 85.1%, a training error of 0.305, and a validation error of 0.455. Although Run 1 ($\alpha = 0.9$) achieved nearly identical validation performance, Run 2 demonstrated slightly lower training error, suggesting marginally more stable optimisation. The small performance differences across high-α settings imply that our chosen λ favours L1-heavy regimes; however, the balanced configuration ($\alpha = 0.5$) and L2-biased configuration ($\alpha = 0.25$) show a clear drop in performance. A more fine-grained sweep, particularly for $\alpha \in [0.7, 0.9]$, could reveal whether even stronger results are attainable.

Test Set Performance The best model—Dropout with inclusion probability 0.97—achieved a test accuracy of 84.7% and a test error of 0.479. These results closely match the model's validation performance, confirming that it generalises well and has not overfitted to the validation set.

Summary Overall, Dropout with a high inclusion probability (0.97) proved to be the most effective regularisation method for this task, yielding the highest validation accuracy and lowest validation error. Elastic Net with a strong L1 bias also performed competitively, suggesting that sparsity plays a beneficial role in this classification problem. Together, these experiments indicate that regularisation methods which emphasise feature selection (L1-like behaviour) tend to generalise best under the given architecture and dataset.

4.1. Custom Activation Function

The results in Figure 7 indicate that the model with the custom activation function struggles to learn effectively. The accuracy, and error curves behave very erratically. The train-

ing accuracy remains low, and the training error decreases, but not significantly, over epochs, suggesting that the model is not fitting the training data well. This poor performance is likely due to vanishing gradients, as evidenced by the gradient flow plot, which shows that gradients diminish across layers during backpropagation. This issue prevents effective weight updates, leading to stagnation in learning. Consequently, the model does not exhibit overfitting; instead, it fails to learn from the training data altogether. It is also worth noting that the experiment was only run for 10 epochs, which may not be sufficient for convergence, but the gradient issues suggest deeper problems with the activation function choice.

5. Conclusion

The analysis presented in this report confirmed that increasing model capacity, whether through width or depth, improves training fit but consistently leads to overfitting on the EMNIST dataset, as evidenced by the widening generalisation gap and rising validation error. This behaviour aligns with the classical bias–variance trade-off described by Goodfellow et al. (Goodfellow et al., 2016)

Our experiments demonstrated that several regularisation techniques can mitigate this effect. Both Elastic Net (with $\lambda = 5 \times 10^{-5}$ and $\alpha \in [0.75, 0.90]$) and Dropout with a high inclusion probability ($p = 0.97$) achieved the strongest generalisation performance, improving validation accuracy and reducing the generalisation gap relative to the baseline. Label smoothing also yielded a modest improvement, although it was less effective than L2-based methods. In contrast, L1 regularisation alone performed poorly; however, its strong contribution within the Elastic Net configurations suggests that sparsity offers benefits for EMNIST, whose images contain substantial redundancy and locally correlated features.

The failure of the custom activation function further highlighted that poor learning dynamics are not always attributable to overfitting. In this case, vanishing gradients prevented the model from learning, resulting in uniformly low accuracy and stagnant optimisation.

Future work should involve a more comprehensive grid search over the Elastic Net hyperparameters (λ, α) and explore combined regularisation approaches—for example, pairing L2 penalties with light Dropout—to determine whether complementary mechanisms can further improve generalisation performance.

References

Goodfellow, Ian, Bengio, Yoshua, and Courville, Aaron. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

Ng, Andrew Y. Feature selection, l1 vs. l2 regularization, and rotational invariance. In *Proceedings of the twenty-first international conference on Machine learning*, pp. 78, 2004.

Srivastava, Nitish, Hinton, Geoffrey, Krizhevsky, Alex, Sutskever, Ilya, and Salakhutdinov, Ruslan. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1): 1929–1958, 2014.

Zou, Hui and Hastie, Trevor. Zou h, hastie t. regularization and variable selection via the elastic net. j r statist soc b. 2005;67(2):301-20. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 67:301 – 320, 04 2005. doi: 10.1111/j.1467-9868.2005.00503.x.